APUNTES. Orientación a Objetos

Programación web con .NET (IFCD59) – Módulo 2: conceptos Programación Orientada a Objetos.

Herencia, Encapsulamiento y Polimorfismo

Base: clase, objeto, atributos y métodos

- Clase: el “molde” (define estructura y comportamiento).
- Objeto: una instancia concreta de la clase.
- Atributos: datos/estado.
- Métodos: acciones/comportamiento.
- Constructor: asegura que el objeto nace en un estado válido.

Ejemplo mínimo

```
public class Persona
{
    public string Nombre { get; set; }
    public int Edad { get; set; }


    // Constructor. Se llama igual que la clase

    // Si no lo creas, se suele crear de forma automática.

    // Si no defines ninguno, C# crea automáticamente un constructor vacío:
    public Persona(string nombre, int edad)
    {
        Nombre = nombre;
        Edad = edad;
    }

    public void Saludar()
    {
        Console.WriteLine($"Hola, soy {Nombre} y tengo {Edad} años");
    }
}

Uso:

var p = new Persona("Ana", 30);
p.Saludar();
```

Idea clave: ¿Qué es la Programación Orientada a Objetos (POO)?

La POO es una forma de programar donde intentamos modelar el mundo real con “objetos”.
Cada objeto combina:

• Datos (estado): lo que el objeto “tiene”.
• Métodos (comportamiento): lo que el objeto “hace”.

Ejemplo mental: un jugador tiene un nombre y una puntuación (datos), y puede sumar puntos o mostrarse (métodos).

Ejemplo rápido: Jugador (primera versión, sencilla)

```
public class Jugador
{
    public string Nombre { get; set; }
    public int Puntuacion { get; set; }

    public void Mostrar()
        => Console.WriteLine($"Jugador: {Nombre} -> ({Puntuacion}
puntos)");
}
```

Esta clase está bien para empezar. Pero… ¿qué pasa si alguien hace Puntuacion = -999? Ahí entra el encapsulamiento.

1) Encapsulamiento (explicación fácil)

Encapsulamiento = “proteger el interior del objeto y controlar cómo se modifica”.

Piensa en un coche real: tú no tocas directamente el motor con las manos para acelerar. Usas un pedal (una interfaz controlada). El coche decide cómo acelerar sin romperse.

En programación, el objeto no debería permitir que cualquiera cambie sus datos internos de cualquier manera. En su lugar, damos métodos/propiedades que aplican reglas y validaciones.

1.1) ¿Qué problema resuelve? (Ejemplo Jugador)

Versión NO encapsulada (demasiado libre):

```
var j = new Jugador();
j.Nombre = "Ana";
j.Puntuacion = -50;  // ¿Tiene sentido? Normalmente NO
```

Si tu juego no permite puntuaciones negativas, este código no debería ser posible. Con encapsulamiento, evitamos estados inválidos.

1.2) Versión encapsulada: controlar cambios con private set + métodos

```
public class Jugador
{
    public string Nombre { get; private set; }
    public int Puntuacion { get; private set; }

    public Jugador(string nombre)
    {
        if (string.IsNullOrWhiteSpace(nombre))
            throw new ArgumentException("El nombre es obligatorio.");

        Nombre = nombre;
        Puntuacion = 0;
    }

    public void SumarPuntos(int puntos)
    {
        if (puntos < 0)
            throw new ArgumentException("Para restar puntos usa
Penalizar().");

        Puntuacion += puntos;
    }

    public void Penalizar(int puntos)
    {
        if (puntos < 0) throw new ArgumentException("La penalización
debe ser positiva.");
        Puntuacion -= puntos; // aquí sí permitimos bajar
    }

    public void Mostrar()
        => Console.WriteLine($"Jugador: {Nombre} -> ({Puntuacion}
puntos)");
}
```

Fíjate en la diferencia:

- Desde fuera NO puedes hacer Jugador.Puntuacion = ... porque el set es privado.
- Para cambiar la puntuación debes usar SumarPuntos() o Penalizar(), que controlan reglas.

Eso es encapsular: el objeto “protege” su estado.

1.3) Campo privado + propiedad (cuando necesitas lógica extra)

A veces se usa un campo privado para guardar el dato y una propiedad pública para exponerlo. Esto es típico cuando quieres validar en el set o hacer cálculos.

```
public class Termostato
{
    private int _temperatura;          // estado interno (campo
privado)

    public int Temperatura            // “puerta controlada”
    {
```

```
        get => _temperatura;
        set
        {
            if (value < 10 || value > 30)
                throw new ArgumentException("Temperatura fuera de
rango (10..30).");

            _temperatura = value;
        }
    }
}
```

1.4) Resumen de encapsulamiento

- No dejar datos importantes “libres” (evitar set público si rompe reglas).
- Exponer métodos para cambiar el estado aplicando validaciones.
- Usar private/protected para ocultar detalles internos.
- Resultado: objetos más seguros y código más fácil de mantener.

2) Herencia (con Vehículo → Coche, Camión y Moto)

Herencia = crear una clase nueva a partir de otra, reutilizando lo común.

Regla mental: Herencia encaja cuando se cumple claramente “ES-UN”.
• Coche ES-UN Vehículo
• Camión ES-UN Vehículo
• Moto ES-UN Vehículo

La clase base define lo común (marca, modelo, arrancar…), y las hijas añaden lo específico.

2.1) Clase base: Vehiculo (lo común)

```
public class Vehiculo
{
    public string Marca { get; }
    public string Modelo { get; }
    public int VelocidadMax { get; }

    protected bool _motorEncendido;

    public Vehiculo(string marca, string modelo, int velocidadMax)
    {
        if (string.IsNullOrWhiteSpace(marca)) throw new
ArgumentException("Marca obligatoria");
```

```
        if (string.IsNullOrWhiteSpace(modelo)) throw new
ArgumentException("Modelo obligatorio");
        if (velocidadMax <= 0) throw new ArgumentException("Velocidad
máxima inválida");

        Marca = marca;
        Modelo = modelo;
        VelocidadMax = velocidadMax;
    }

    public void Arrancar() => _motorEncendido = true;
    public void Parar() => _motorEncendido = false;

    public virtual void MostrarFicha()
    {
        Console.WriteLine($"Vehículo: {Marca} {Modelo} | Vmáx:
{VelocidadMax} km/h");
    }
}
```

2.2) Clases hijas (lo específico de cada tipo)

```
public class Coche : Vehiculo
{
    public int NumeroPuertas { get; }

    public Coche(string marca, string modelo, int velocidadMax, int
numeroPuertas)
        : base(marca, modelo, velocidadMax)
    {
        NumeroPuertas = numeroPuertas;
    }

    public override void MostrarFicha()
    {
        Console.WriteLine($"Coche: {Marca} {Modelo} | Puertas:
{NumeroPuertas} | Vmáx: {VelocidadMax} km/h");
    }
}

public class Camion : Vehiculo
{
    public int CargaMaxKg { get; }

    public Camion(string marca, string modelo, int velocidadMax, int
cargaMaxKg)
        : base(marca, modelo, velocidadMax)
    {
        CargaMaxKg = cargaMaxKg;
    }

    public override void MostrarFicha()
    {
```

```
        Console.WriteLine($"Camión: {Marca} {Modelo} | Carga:
{CargaMaxKg} kg | Vmáx: {VelocidadMax} km/h");
    }
}

public class Moto : Vehiculo
{
    public int CilindradaCc { get; }

    public Moto(string marca, string modelo, int velocidadMax, int
cilindradaCc)
        : base(marca, modelo, velocidadMax)
    {
        CilindradaCc = cilindradaCc;
    }

    public override void MostrarFicha()
    {
        Console.WriteLine($"Moto: {Marca} {Modelo} | {CilindradaCc} cc
| Vmáx: {VelocidadMax} km/h");
    }
}
```

2.3) ¿Qué ganas con herencia? (en claro)

- Reutilizas código común en la clase base (no repites Marca/Modelo/Arrancar/Parar).
- Las hijas solo se preocupan de lo que las hace diferentes.
- Si mañana cambias una regla común (por ejemplo validar Marca), lo haces una vez en Vehiculo.

2.4) Error típico: usar herencia cuando NO es “ES-UN”

Ejemplo: “Un vehículo tiene un motor” (TIENE-UN), no “es un motor”.
Ahí suele ser mejor composición (una clase dentro de otra).

```
public class Motor
{
    public int Caballos { get; }
    public Motor(int caballos) => Caballos = caballos;
}

public class VehiculoConMotor
{
    public string Marca { get; }
    public Motor Motor { get; }

    public VehiculoConMotor(string marca, Motor motor)
    {
        Marca = marca;
        Motor = motor;
    }
}
```

3) Polimorfismo

Polimorfismo = “mismo método, resultados diferentes según el tipo real del objeto”.

Ejemplo: todos los vehículos saben “MostrarFicha()”. Pero cada tipo lo muestra a su manera.
Eso permite escribir código genérico (por ejemplo un foreach) sin if/else grandes.

3.1) Polimorfismo dinámico (virtual + override)

Ejemplo con lista de Vehiculo:

```
var flota = new List<Vehiculo>
{
    new Coche("Toyota", "Corolla", 180, 5),
    new Camion("Volvo", "FH", 140, 18000),
    new Moto("Yamaha", "MT-07", 200, 689)
};

foreach (Vehiculo v in flota)
{
    v.MostrarFicha(); // cada objeto ejecuta SU versión
}
```

Esto funciona porque MostrarFicha() es virtual en Vehiculo y override en las hijas.
Si no fuera virtual/override, siempre se llamaría la versión del padre.

3.2) Un método que recibe Vehiculo (código reutilizable)

```
public void ImprimirFicha(Vehiculo v)
{
    v.MostrarFicha(); // sirve para coche, camión o moto
}
```

3.3) Polimorfismo estático: sobrecarga (overload)

Mismo nombre, distintos parámetros. El compilador decide cuál usar.

```
public class Calculadora
{
    public int Sumar(int a, int b) => a + b;
    public int Sumar(int a, int b, int c) => a + b + c;
    public double Sumar(double a, double b) => a + b;
}
```

Otro ejemplo:

4) Variables privadas (campos) — muy relacionado con encapsulamiento

Un campo privado es una variable dentro de la clase que no se ve desde fuera. Se usa para guardar estado interno y protegerlo.

Ejemplo 1: contador

```
public class Contador
{
    private int _valor;
    public int Valor => _valor;

    public void Incrementar() => _valor++;
    public void Reiniciar() => _valor = 0;
}
```

Ejemplo 2: dado (Random interno)

```
public class Dado
{
    private readonly Random _rnd = new Random();
    public int Tirar() => _rnd.Next(1, 7);
}
```

5) Cómo reconocer cada concepto

- Encapsulamiento: private/protected + métodos para cambiar el estado (no todo con set público).
- Herencia: ": ClaseBase" + base(...). Relación ES-UN.
- Polimorfismo: virtual/override + código que usa la clase base sin conocer el tipo exacto.

6) Ejercicios prácticos (recomendados)

- 1) Encapsulamiento: cambia Jugador para que Puntuacion tenga private set.
- 2) Encapsulamiento: crea SumarPuntos() y Penalizar() con validaciones.
- 3) Herencia: crea Autobus : Vehiculo con propiedad Plazas.
- 4) Polimorfismo: añade Autobus a la List<Vehiculo> y comprueba que el foreach funciona igual.
- 5) Crea un método ImprimirFicha(Vehiculo v) y pásale varios vehículos.